

گزارش جامع پیاده‌سازی، رفع چالش‌ها و راه‌اندازی داشبورد سیستم حسگری راداری وای‌فای (ISAC) با پلتفرم ESpectre

امیرعلی یزدانی

۱۷ شهریور ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه و ساختار کلی پروژه
۳	۲	معماری ساختاری ESPHome و عدم نیاز به کدنویسی مستقیم C++
۳	۳	مراحل آماده‌سازی و نصب بسته‌های پایه
۴	۱۰۳	ایجاد محیط مجازی و نصب ابزار ESPHome
۴	۲۰۳	تعریف متغیرهای دسترسی شبکه (secrets.yaml)
۴	۴	اصلاح فایل‌های پیکربندی سخت‌افزاری و نرم‌افزاری
۴	۱۰۴	اصلاح فایل espectre-s۳.yaml
۴	۲۰۴	اصلاح فایل espectre-s۳-dev.yaml
۴	۵	دستورات پاک‌سازی، کامپایل و مانیتورینگ سریال
۴	۱۰۵	پاک‌سازی کامل حافظه فلش
۴	۲۰۵	کامپایل و فلش فریم‌ورک
۵	۳۰۵	مشاهده لاگ‌های بی‌درنگ
۵	۶	راه‌اندازی Docker و استقرار سرور Home Assistant
۵	۱۰۶	تنظیم آینه‌های رجیستری داکر (Docker Daemon Mirror)
۵	۲۰۶	اجرای کانٹینر Home Assistant با دسترسی به شبکه سیستم
۵	۳۰۶	پیکربندی گره در پنل تحت وب Home Assistant و مدیریت تغییر IP
۶	۴۰۶	ایجاد داشبورد گرافیکی
۶	۷	تحلیل الگوریتم‌های آشکارسازی: MVS در برابر ML
۶	۱۰۷	الگوریتم واریانس متحرک فضایی (MVS)
۷	۲۰۷	الگوریتم یادگیری ماشین (ML)
۷	۸	مفهوم ISAC و تفاوت بنیادین با پلتفرم RuView
۷	۹	خروجی نهایی و داشبورد پایش

۱ مقدمه و ساختار کلی پروژه

پلتفرم ESPectre یک فریم‌ورک سبک و بی‌درنگ برای استخراج اطلاعات وضعیت کانال (Channel State Information - CSI) امواج وای‌فای بر بستر چیپ‌های خانواده ESP32 است. در این فرآیند، استخراج اطلاعات فاز و دامنه ساب‌کریها جهت پایش حضور و شدت تلاطم محیط با استفاده از برد ESP32-S3 (N16R8) صورت پذیرفته و خروجی‌ها در داشبورد گرافیکی Home Assistant یکپارچه‌سازی شده‌اند.

۲ معماری ساختاری ESPHome و عدم نیاز به کدنویسی مستقیم C++

برخلاف پروژه‌های سنتی امبدد، در این فریم‌ورک هیچ نیازی به کدنویسی دستی C++ نیست؛ پلتفرم ESPHome به‌عنوان یک تولیدکننده هوشمند کد (Code-Generator) عمل می‌کند:

۱. متغیرها و ماژول‌ها در فایل‌های مانیفست متنی با فرمت YAML تعریف می‌شوند.
 ۲. موتور ESPHome فایل‌های متنی را خوانده و کدهای بهینه‌شده C++ متناسب با معماری ESP-IDF را تولید می‌کند.
 ۳. بیلدسیستم Ninja/CMake کدها را کامپایل کرده و فایل‌های فریم‌ورک باینری را ایجاد می‌نماید.
- این سازوکار دارای کمترین سربار حافظه (Zero-Bloat) بوده و هسته ESPectre تنها حدود ۷۰ کیلوبایت از فلش و کمتر از ۱۰۰ بایت از رم را مصرف می‌کند.

۳ مراحل آماده‌سازی و نصب بسته‌های پایه

۱.۳ ایجاد محیط مجازی و نصب ابزار ESPHome

```
cd ~/Desktop/Amirali/WifiSensing/espectre/espectre
python3 -m venv venv
source venv/bin/activate
python3 -m pip install esphome
```

۲.۳ تعریف متغیرهای دسترسی شبکه (secrets.yaml)

برای مدیریت امن مشخصات شبکه، فایل secrets.yaml در مسیر نمونه‌ها ایجاد گردید:

```
cat > examples/secrets.yaml << 'EOF'
wifi_ssid: "Sharif-WiFi"
wifi_password: ""
EOF
```

۴ اصلاح فایل‌های پیکربندی سخت‌افزاری و نرم‌افزاری

جهت تطابق فریم‌ور با برد ESP32-S3-N16R8 (دارای ۱۶ مگابایت فلش و ۸ مگابایت Octal PSRAM)، اصلاحات ساختاری زیر اعمال شد:

۱.۴ اصلاح فایل `espectre-s۳.yaml`

فعال سازی حافظه Octal PSRAM و هدایت لاگ ها به USB-JTAG:

```
esp32:
  variant: ESP32S3
  cpu_frequency: 240MHz
  board: esp32-s3-devkitc-1
  framework:
    type: esp-idf
    version: 5.5.1
    sdkconfig_options:
      CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ: "240"
      CONFIG_SPIRAM: "y"
      CONFIG_SPIRAM_MODE_OCT: "y"
      CONFIG_SPIRAM_TYPE_AUTO: "y"
      CONFIG_SPIRAM_SPEED_80M: "y"
      CONFIG_SPIRAM_BOOT_INIT: "y"
      CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG: "y"

psram:
  mode: octal
  speed: 80MHz
```

۲.۴ اصلاح فایل `espectre-s۳-dev.yaml`

تنظیم لاگر سخت افزاری بر روی کنسول USB_SERIAL_JTAG:

```
logger:
  level: DEBUG
  hardware_uart: USB_SERIAL_JTAG
  logs:
    sensor: INFO
    binary_sensor: INFO
```

۵ دستورات پاک سازی، کامپایل و مانیتورینگ سریال

۱.۵ پاک سازی کامل حافظه فلش

```
source venv/bin/activate
esptool erase-flash
```

۲.۵ کامپایل و فلش فریمورک

```
esphome run examples/espectre-s3-dev.yaml
```

در منوی انتخاب پورت، گزینه متناظر با درگاه بومی `/dev/ttyACM0` (عدد ۱) انتخاب شد.

```
esphome logs examples/espectre-s3-dev.yaml
```

۶ راه‌اندازی Docker و استقرار سرور Home Assistant

۱.۶ تنظیم آینه‌های رجیستری داکر (Docker Daemon Mirror)

به دلیل محدودیت‌های دسترسی به رجیستری‌های رسمی، آدرس‌های آینه داخلی درون فایل پیکربندی داکر تنظیم شدند:

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json << 'EOF'
{
  "registry-mirrors": [
    "https://docker.arvancloud.ir",
    "https://registry.docker.ir"
  ]
}
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
```

۲.۶ اجرای کانینر Home Assistant با دسترسی به شبکه سیستم

به دلیل تفاوت درگاه شبکه کابل LAN سیستم و وای‌فای برد، کانینر در حالت `-net=host` مستقر گردید تا ارتباطات لایه شبکه محلی مسدود نگردد:

```
docker run -d --name homeassistant --privileged --restart=unless-stopped
--net=host -e TZ=Asia/Tehran -v ~/homeassistant_config:/config
ghcr.io/home-assistant/home-assistant:stable
```

۳.۶ پیکربندی گره در پنل تحت وب Home Assistant و مدیریت تغییر IP

۱. ورود به پنل از طریق مرورگر با آدرس <http://localhost:8123>.

۲. ورود با نام کاربری و رمز `username: amirali, password: 123456789`.

۳. ورود به مسیر: `Add Integration → Devices & Services → Settings`.

۴. انتخاب مازول ESPHome و وارد کردن دستی آدرس آی‌پی برد (به عنوان مثال: 172.27.4.45) روی پورت پیش فرض ۶۰۵۳.

مدیریت تغییر آدرس IP دینامیک (**DHCP Re-assignment**): به دلیل استفاده از پروتکل DHCP در روترها و شبکه‌های محلی، ممکن است پس از هر بار خاموش/روشن شدن برد یا روتر، آدرس IP اختصاص یافته تغییر کرده و در نتیجه موجودیت‌ها و سنسورها در داشبورد Home Assistant در وضعیت ناموجود (Unavailable) قرار گیرند. برای رفع این مشکل، مراحل زیر دنبال می‌شود:

- استخراج آدرس IP جدید: ابتدا از طریق اتصال کابل USB و اجرای دستور زیر، لاگ‌های زنده برد خوانده شده و آدرس جدید در خطوط مربوط به اتصال وای‌فای (مقابل عبارت WiFi Connected! IP Address:) استخراج می‌گردد:

```
esphome logs examples/espectre-s3-dev.yaml
```

- پیکربندی مجدد در **Home Assistant**: بدون نیاز به حذف موجودیت‌ها یا ساخت داشبورد مجدد، در پنل Home Assistant به مسیر Settings → Devices & Services رفته، بر روی کارت مربوط به ESPHome و منوی سه‌نقطه گره کلیک کرده و گزینه **Reconfigure** انتخاب می‌شود. با وارد کردن آدرس IP جدید و تایید آن، تمام سنسورها و نمودارهای داشبورد بلافاصله به وضعیت فعال (Active) بازمی‌گردند.

۴.۶ ایجاد داشبورد گرافیکی

پس از ایجاد یک داشبورد جدید، ویرایشگر ساختار خام (Raw configuration editor) باز شده و محتویات فایل آماده examples/home-assistant-dashboard-dev.yaml درون آن بارگذاری شد تا گنج‌های شدت حرکت، وضعیت آشکارساز و نمودارهای سابقه ۲۴ ساعته فعال گردند. برای این کار باید به بخش Map در نوار سمت چپ رفته و روی علامت مداد بالا سمت راست بزنید.

هشدار مهم در خصوص درگاه اتصال برد (**Hardware Port Warning**): بردهای توسعه ESP32-S3 معمولاً دارای دو درگاه فیزیکی Type-C مجزا هستند. برای فلش، مانیتورینگ زنده سریال و ارتباط مستقیم با کنترلر داخلی (USB-JTAG (ttyACM0)، کابل USB باید حتماً به درگاه سمت چپ برد متصل گردد؛ در غیر این صورت فریم‌ورک شناسایی نشده و ارسال دستورات فلش با خطا مواجه خواهد شد. نکته: برای تشخیص حرکت از دو الگوریتم ml و mvs استفاده می‌شود. در حالت پیشفرض از mvs استفاده می‌شود. برای تغییر این باید در فایل Desktop/Amirali/WifiSensing/espectre/espectre/examples/espectre-s3-dev.yaml خط ۲۴ تغییر ایجاد شود.

۷ تحلیل الگوریتم‌های آشکارسازی: MVS در برابر ML

۱.۷ الگوریتم واریانس متحرک فضایی (MVS)

این روش آماری، با تحلیل تغییرات تلاطم میدان امواج نسبت به حالت سکون عمل می‌کند:

$$\text{Var}(s) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

در ۱۳ ثانیه اول پس از راه‌اندازی، سیستم در حالت سکون دو فاز Gain Lock و کالیبراسیون باند بهینه (NBVI) را اجرا کرده و ۱۲ ساب‌کریپ با بالاترین سیگنال به نوبت را انتخاب می‌نماید.

۲۰۷ الگوریتم یادگیری ماشین (ML)

این روش از یک شبکه عصبی پرسپترون چندلایه تعبیه شده (MLP) با ساختار $1 \rightarrow 16 \rightarrow 32 \rightarrow 9$ بهره می برد. زمان راه اندازی آن کمتر از ۳ ثانیه بوده و به کالیبراسیون نیازی ندارد، اما از ساب کپریهای ثابت از پیش تعیین شده استفاده می کند. با تغییر پارامتر `ml: detection_algorithm` در فایل کانفیگ و فلش مجدد، سیستم به حالت عصبی منتقل می شود.

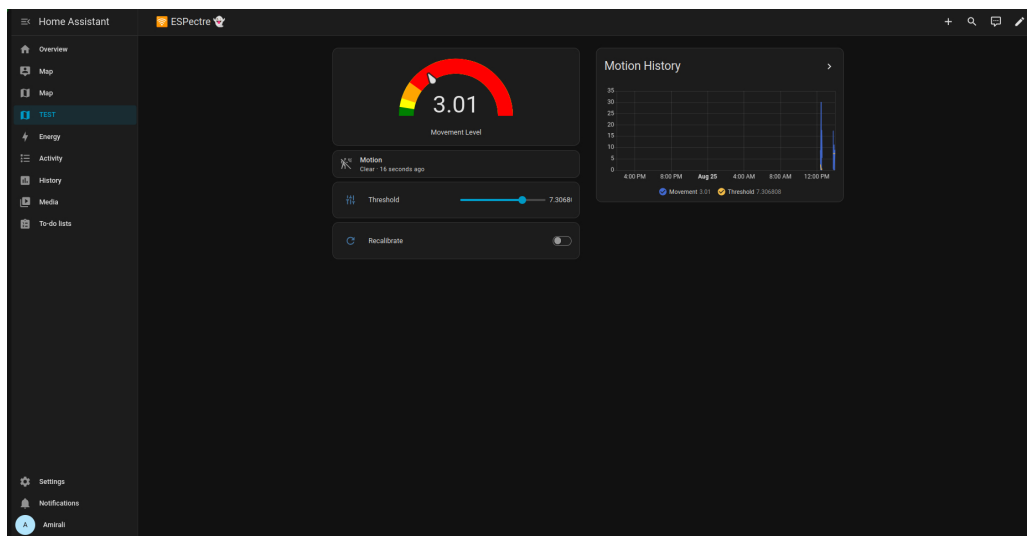
۸ مفهوم ISAC و تفاوت بنیادین با پلتفرم RuView

مفهوم مخبرات مجتمع و حسگری (ISAC) به بهره گیری هم زمان از سیگنال های ارتباطی استاندارد جهت حسگری محیطی اطلاق می گردد:

- پلتفرم RuView: داده های خام حسگری را در ساختار مش چندنوده (Multi-Static) بسته بندی کرده و برای نتایج به یک سرور اختصاصی خارجی (Rust Server) مخبره می کند.
- پلتفرم ESpectre: مفهوم ISAC به صورت متمرکز درون گره پیاده شده است؛ تراشه با ارسال ترافیک استاندارد شبکه (Ping/DNS)، اطلاعات CSI امواج برگشتی را هم زمان استخراج و پردازش های آماری/عصبی را بدون نیاز به سرور محاسباتی خارجی در همان لبه سخت افزار اجرا می نماید.

۹ خروجی نهایی و داشبورد پایش

پس از انجام موفقیت آمیز مراحل، داشبورد تحلیلی در بستر Home Assistant مستقر گردید که وضعیت بی درنگ تلاطم امواج، نمرات حرکت و سوابق حسگری را مصور سازی می نماید:



شکل ۱: داشبورد نهایی پایش اطلاعات حسگری وای فای در Home Assistant

۱۰ نتیجه‌گیری

در این پروژه، سیستم حسگری ESpectre با موفقیت بر روی برد ESP32-S3 کامپایل و پایداری سازی گردید. با غلبه بر چالش‌های لایه فیزیکی پورت سریال، تنظیم دسترسی‌های کانتینر داکر و تطبیق فرکانس رادیویی، ارتباط بی‌درنگ با Home Assistant برقرار شد و قابلیت‌های الگوریتم‌های MVS و ML در سنجش تلاطم فضایی امواج وای‌فای تثبیت گردید.